



PARADIGMAS DE PROGRAMACIÓN

UNIDAD 4

Dr. Pablo Javier Vidal

Facultad de Ingenieria
Universidad Nacional de Cuyo

2026

- 1 Introducción
- 2 Manejo de memoria
- 3 Parametros
- 4 Punteros
- 5 Puntero a función

Introducción

De código fuente a ejecución

Flujo conceptual:

- 1 Conceptos basicos - Paradigmas
- 2 Código fuente
- 3 Análisis léxico
- 4 Análisis sintáctico
- 5 AST
- 6 Tabla de símbolos
- 7 Análisis semántico
- 8 Generación de código
- 9 Runtime

Los subprogramas (procedimientos, metodos, funciones) son un punto clave en este proceso.

Ejemplo en distintos paradigmas

```
int suma(int a,int b){  
    return a+b;  
}
```

```
suma a b = a + b
```

```
suma(A,B,R):- R is A+B.
```

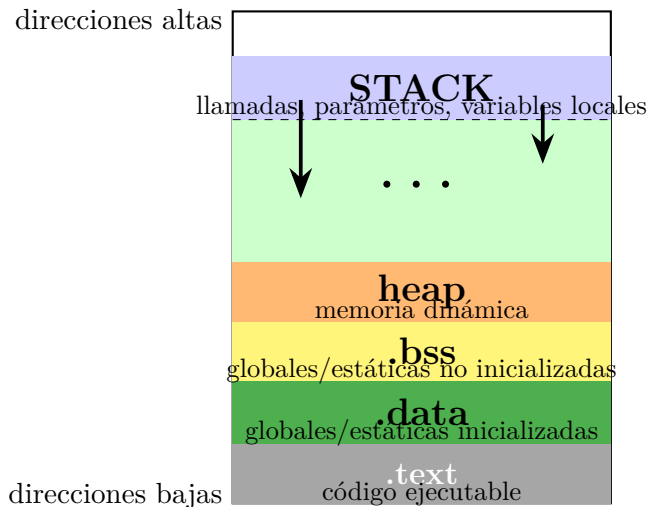
Mismo concepto, distintas semánticas.

Manejo de memoria

Idea general

- Un proceso en ejecución utiliza regiones de memoria con funciones distintas.
- No toda la memoria cumple el mismo papel: una parte almacena instrucciones, otra datos persistentes, otra memoria dinámica y otra contexto de llamadas.
- La figura siguiente nos sirve como mapa conceptual para entender la organización clásica de memoria de un programa.

Mapa general de memoria del proceso



¿Qué se almacena en cada región?

Región	Contenido típico
.text	Instrucciones del programa ya traducidas a código máquina o a una representación ejecutable.
.data	Variables globales o estáticas con valor inicial explícito.
.bss	Variables globales o estáticas sin inicialización explícita; el sistema las reserva antes de ejecutar.
Heap	Objetos y estructuras creadas dinámicamente durante la ejecución: listas, árboles, objetos, buffers, etc.
Stack	Registros de activación: parámetros, dirección de retorno, variables locales, referencias temporales y control de llamadas.

Idea clave

Las regiones `.text`, `.data` y `.bss` suelen tener tamaño más predecible; heap y stack cambian dinámicamente durante la ejecución.

Ejemplo corto para ubicar código y datos

```
#include <stdio.h>

int contador = 10;    // .data
int total;           // .bss

int cuadrado(int x) { // .text
    int y = x * x;    // stack
    return y;
}

int main(void) {
    int r = cuadrado(4); // stack
    int *p = malloc(sizeof(int));
    *p = r;              // heap
    printf("%d\n", *p);
    return 0;
}
```

- La función `cuadrado` forma parte del código ejecutable.
- `contador` va a `.data` porque nace inicializada.
- `total` va a `.bss` porque no tiene valor inicial explícito.
- `r` y `y` son variables locales: viven en el **stack**.
- El entero apuntado por `p` se reserva en el **heap**.

Relación entre heap y stack

Stack

- Administración automática.
- Muy rápido para reservar y liberar.
- Sigue disciplina LIFO.
- Ideal para llamadas a funciones y datos locales de vida corta.

Heap

- Administración dinámica en tiempo de ejecución.
- Permite tamaños flexibles y vida útil no ligada al bloque actual.
- Requiere malloc/free, new/delete o recolector de basura.
- Es más flexible, pero más costoso de gestionar.

Posible problema

Si el stack crece hacia abajo y el heap hacia arriba, ambos compiten por el espacio disponible del proceso.

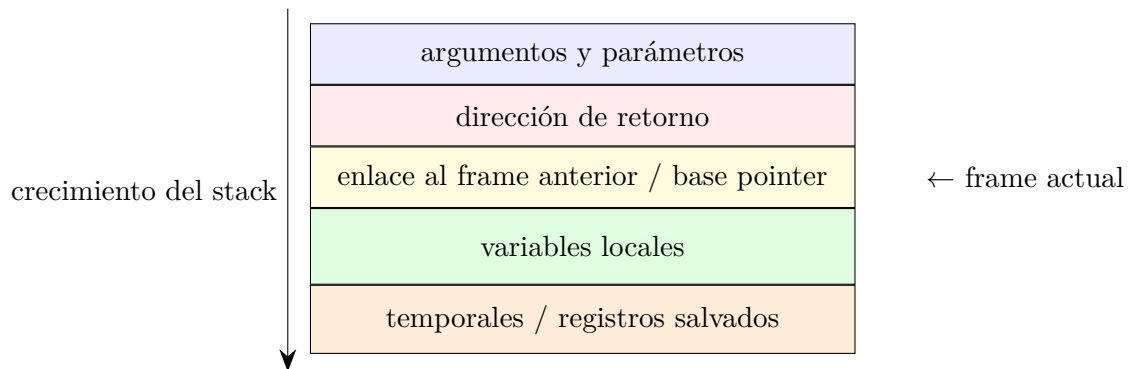
Campos típicos de un stack frame

- **Parámetros:** valores recibidos por la función.
- **Dirección de retorno:** a qué instrucción volver cuando la función finaliza.
- **Enlace dinámico o frame pointer anterior:** permite reconstruir la cadena de llamadas.
- **Variables locales:** datos declarados dentro de la función.
- **Temporales y registros guardados:** resultados intermedios o registros que deben preservarse.

Observación

La organización exacta depende de la arquitectura, del compilador y del lenguaje, pero la idea general se mantiene.

¿Qué es un registro de activación o stack frame?



- Es la estructura que representa una activación concreta de una función.
- Dos llamadas a la misma función producen dos frames distintos.
- Contiene tanto información de **control** como datos **locales**.

Ejemplo de función y activaciones sucesivas

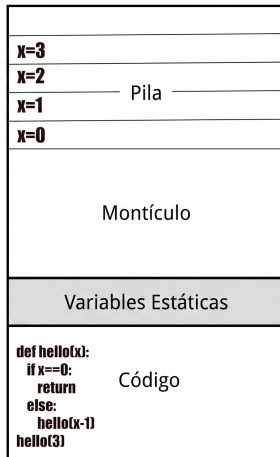
```
def hello(x):  
    if x == 0:  
        return  
    else:  
        hello(x - 1)
```

hello(3)

- hello(3) crea un frame con x=3.
- Luego hello(2) agrega otro frame.
- Después hello(1) y hello(0).
- Al llegar al caso base, los frames se retiran en orden inverso.

Entrada al stack: llamadas a funciones

```
def hello(x):
    if x==0:
        return
    else:
        hello(x-1)
hello(3)
```



Pila

Montículo

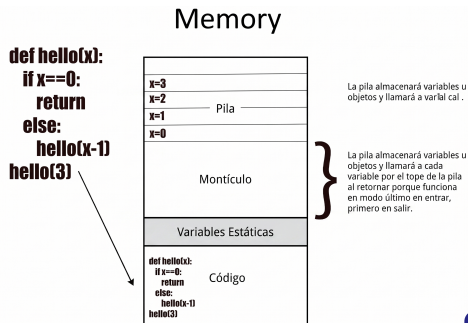
Variables Estáticas

Código

La pila almacenará variables u objetos y llamará a varial cal .

La pila almacenará variables u objetos y llamará a cada variable por el tope de la pila al retornar porque funciona en modo último en entrar, primero en salir.

Entrada al stack: llamadas a funciones

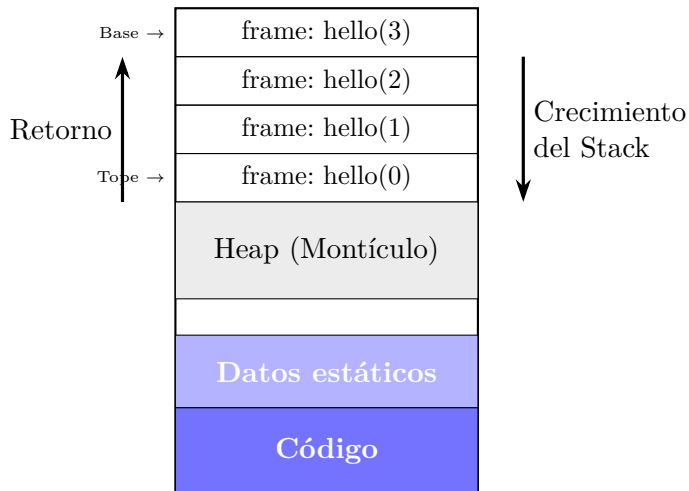


- Cada llamada a una función crea un nuevo contexto de ejecución.
- Ese contexto se apila sobre los anteriores.
- Cuando la función termina, su contexto se desapila.
- Por eso el stack modela muy bien llamadas anidadas y recursivas.

Idea operativa

Llamar implica **apilar**; retornar implica **desapilar**.

Evolución del stack en la recursión



Secuencia típica de llamada y retorno

Lectura

En este modelo, el **Stack** comienza en direcciones altas y crece hacia el **Heap**. El último frame en entrar (`hello(0)`) queda en el tope inferior.

- 1 El programa ejecuta una instrucción de llamada.
- 2 Se guardan datos de control, como la dirección de retorno.
- 3 Se crea el nuevo stack frame para la función invocada.
- 4 La función usa sus parámetros y variables locales.
- 5 Al terminar, deja el valor de retorno si corresponde.
- 6 El frame se elimina y la ejecución vuelve al punto guardado.

Consecuencia

Si una función recursiva no alcanza su caso base, el stack sigue creciendo y puede producirse *stack overflow*.

Resumen final

- El proceso organiza su memoria en regiones con funciones bien diferenciadas.
- El código vive en `.text`; los datos estáticos en `.data` y `.bss`.
- El heap resuelve la memoria dinámica de vida flexible.
- El stack administra llamadas, variables locales y control de ejecución.
- El stack frame es la pieza central para entender cómo una función existe durante su ejecución.

Comprender esta organización ayuda a explicar **recursión, paso de parámetros, alcance, rendimiento y errores de memoria**.

Parametros

Parámetros

Dos tipos:

- formales
- reales

Sintaxis vs semántica:

- cómo se escriben
- cómo se evalúan
- cómo se almacenan

Mecanismos de pasaje

- por valor
- por referencia
- por nombre
- copy-restore
- resultado

Impactan en:

- aliasing
- eficiencia
- efectos laterales

Pasaje por valor

```
void f(int x){  
    x=10;  
}  
int a=5;  
f(a);
```

Resultado:
a sigue valiendo 5.

Pasaje por referencia

```
void f(int &x){  
    x=10;  
}
```

Se modifica la variable original.

Pasaje por nombre

Evaluación diferida.

Cada uso evalúa la expresión nuevamente.

Relacionado con:

- macros
- lazy evaluation

Parámetros anónimos

```
lambda x: x+1
```

```
\x -> x+1
```

Permiten:

- funciones inline
- programación funcional

Subprogramas como parámetros

```
map (+1) [1,2,3]
```

```
stream.map(x->x+1)
```

Funciones de orden superior.

Closures

Un closure captura:

- variables libres
- ambiente léxico

Relaciona:

- scope
- lambda calculus
- activation record

Closure ejemplo

```
function f(x){  
  return function(y){  
    return x+y;  
  }  
}
```

La función interna recuerda el valor de x.

Lambda calculus

Base formal de subprogramas.

Conceptos:

- abstracción
- aplicación
- variables libres
- reducción beta

Ejemplo:

$$(\lambda x.x + 1)5$$

Idea general

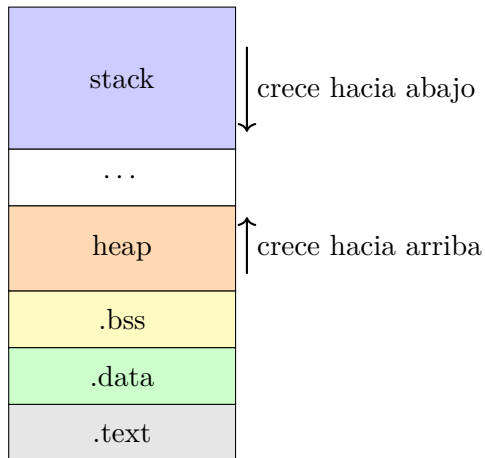
Cuando se llama a una función:

- Se crea un nuevo **stack frame**
- Los parámetros se almacenan en el **stack frame** (espacio de registro de activación)
- El tipo de pasaje determina:
 - si se copia el valor
 - si se copia la dirección
 - si se comparte memoria

Conceptos clave:

- Stack = memoria automática
- Heap = memoria dinámica
- Dirección de memoria = referencia a un dato

Estructura general de memoria



Stack frame (registro de activación)

Cada llamada crea un bloque de memoria en el stack:

parámetros
dirección retorno
variables locales
temporales

Cada llamada genera un nuevo frame.

Pasaje por valor

El valor se copia en el stack frame.

```
void f(int x)
```

```
{
```

```
    x = 20;
```

```
}
```

```
int main()
```

```
{
```

```
    int a = 10;
```

```
    f(a);
```

```
}
```

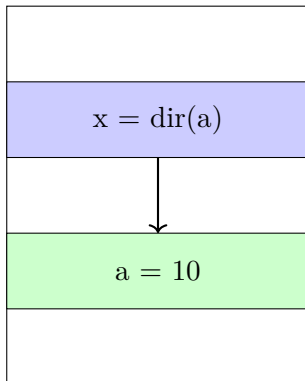
Pasaje por referencia (punteros)

Se pasa la dirección de memoria.

```
void f(int *x)
{
    *x = 20;
}

int main()
{
    int a = 10;
    f(&a);
}
```

Memoria en pasaje por referencia



El parámetro contiene la dirección de memoria.
Ambas variables apuntan al mismo dato.

Parámetros y heap

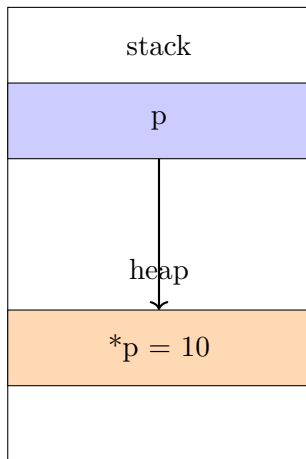
```
void f(int *x)
{
    *x = 30;
}

int main()
{
    int *p;
    p = malloc(sizeof(int));

    *p = 10;

    f(p);
}
```

Relación stack y heap



El puntero está en el stack. El dato está en el heap.

Arreglos como parámetros

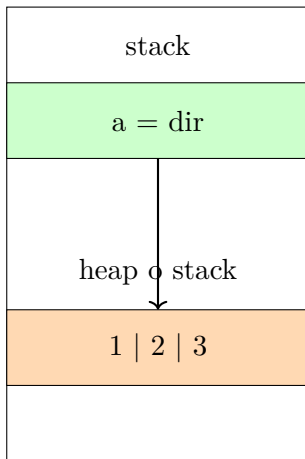
```
void f(int v[])
{
    v[0] = 50;
}

int main()
{
    int a[3] = {1,2,3};

    f(a);
}
```

Los arreglos se pasan como dirección de memoria.

Arreglos en memoria



Comparación

tipo	que se copia	efecto cambios
valor	dato	no modifica original
referencia	dirección	modifica original
puntero a heap	dirección	modifica heap
array	dirección	modifica original

Caso de analisis

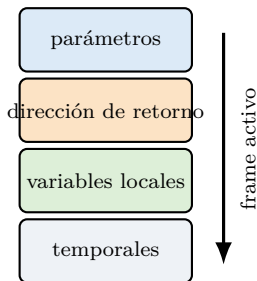
```
1 def hello(x):
2     if x == 1:
3         return "op"
4     else:
5         u = 1
6         e = 12
7         s = hello(x-1)
8         e += 1
9         print(s)
10        print(x)
11        u += 1
12    return e
13
14 hello(4)
```

Idea central: ¿qué es un stack frame?

Resumen conceptual

Un **stack frame** es el bloque de datos asociado a **una invocación concreta** de una función.

- parámetros/argumentos,
- dirección de retorno,
- variables locales,
- temporales necesarios para continuar la ejecución.

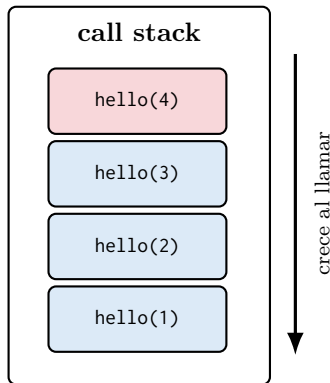


Del código al stack: secuencia de llamadas

- `hello(4)` llama a `hello(3)`
- `hello(3)` llama a `hello(2)`
- `hello(2)` llama a `hello(1)`
- `hello(1)` alcanza el caso base y retorna

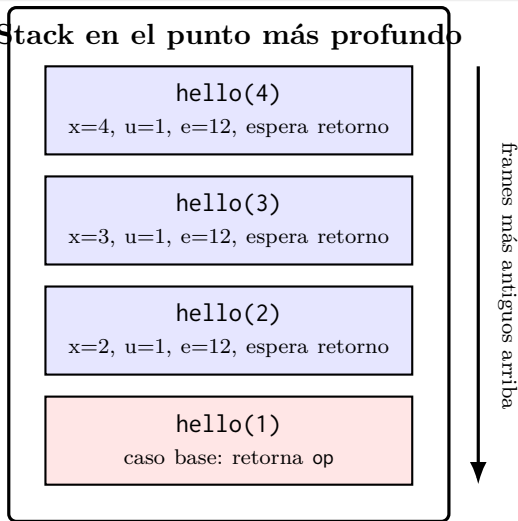
Observación

Cada llamada pendiente necesita conservar su propio contexto. Ese contexto es, justamente, su **stack frame**.



Asignación: cómo se apilan las llamadas

Stack en el punto más profundo



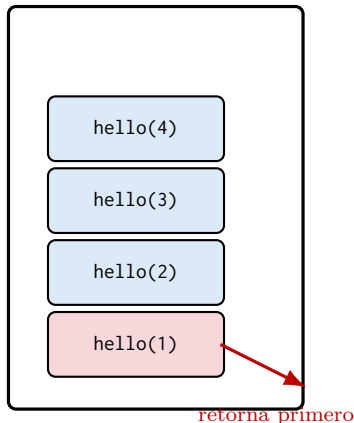
Siguiendo el flujo de ejecución...

Caso base: el frame inferior retorna primero

Lectura didáctica de la página

Cuando $x == 1$, la función satisface la condición de retorno y el frame activo se desapila inmediatamente.

- El tope del stack (abajo) es `hello(1)`.
- Ese frame no necesita seguir avanzando por el `else`.
- Al retornar, el frame superior vuelve a quedar activo.



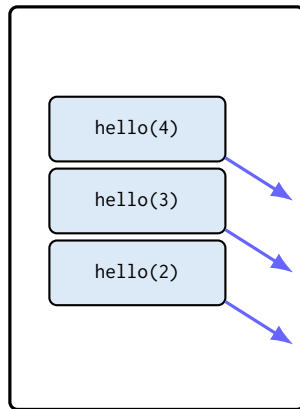
Esta es la idea expresada explícitamente: al cumplirse el *base condition*, el frame actual (en el tope inferior) retorna y el anterior pasa a ser el activo.

Desasignación: el stack se vacía en orden LIFO

- Primero vuelve `hello(1)` (estaba al fondo).
- Después continúa `hello(2)`.
- Luego `hello(3)`.
- Finalmente `hello(4)`.

Consecuencia

Los frames se desapilan en el orden inverso al que fueron apilados.



En la página se remarca que la devolución ocurre en orden inverso al de asignación: comportamiento LIFO (*last in, first out*).

Qué ocurre al volver: trazado de ejecución

Una vez que el caso base devuelve `op`, cada frame pendiente reanuda la ejecución justo después de la llamada recursiva:

```
hello(1) retorna "op"  
hello(2): s = "op" ; e = 13 ; imprime "op" y 2 ; retorna 13  
hello(3): s = 13    ; e = 13 ; imprime 13    y 3 ; retorna 13  
hello(4): s = 13    ; e = 13 ; imprime 13    y 4 ; retorna 13
```

Salida visible

```
op    2    13    3    13    4
```


Cierre

- 1 Una llamada activa = un stack frame.
- 2 La recursión crea varios frames de la misma función.
- 3 El caso base permite empezar a desapilar.
- 4 El retorno ocurre en orden LIFO.
- 5 El frame conecta el nivel del lenguaje con el nivel máquina.

Idea final

Entender el stack frame es entender **qué necesita recordar una función** para poder detenerse, llamar a otra y luego continuar exactamente donde estaba.

Punteros

Motivación

Para que un programa pueda ejecutarse, el procesador necesita información sobre:

- qué instrucción ejecutar
- dónde están los datos actuales
- cuál es el contexto de la función en ejecución

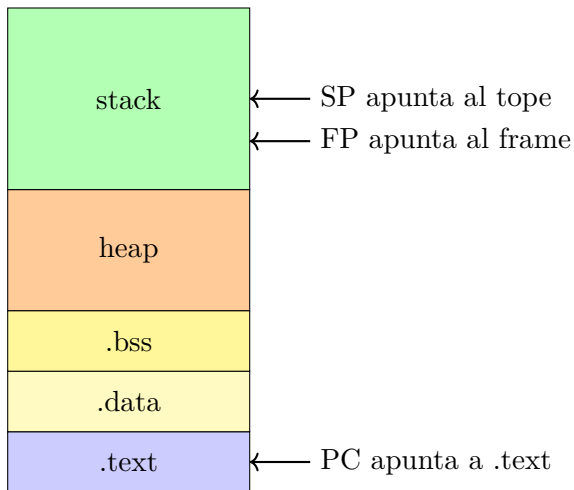
En el nivel máquina aparecen tres registros clave:

- PC (Program Counter)
- SP (Stack Pointer)
- FP (Frame Pointer)

Estos registros conectan:

- llamadas a funciones
- stack frames
- ejecución del programa

Relación con la memoria del proceso



Program Counter (PC)

El Program Counter contiene:

- dirección de la próxima instrucción
- ubicación dentro del segmento .text

Funciones principales:

- avanzar secuencialmente
- saltar en estructuras de control
- guardar dirección de retorno en llamadas

```
int suma(int a,int b){  
    return a+b;  
}
```

El PC cambia cuando ocurre:

- call, return, if, while

Stack Pointer (SP)

El Stack Pointer apunta al tope del stack.

El stack contiene:

- variables locales
- parámetros
- dirección de retorno
- temporales

Cada llamada a función:

- decrementa SP
- reserva memoria

Al retornar:

- incrementa SP
- libera memoria

Frame Pointer (FP)

El Frame Pointer permite acceder al contexto actual de ejecución.

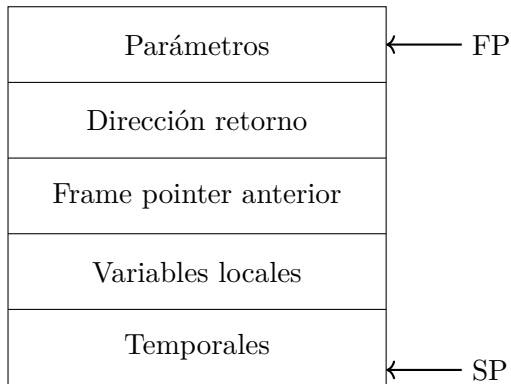
Ventajas:

- referencia estable dentro del frame
- facilita acceso a parámetros
- permite recorrer la cadena de llamadas

Relación:

- SP cambia dinámicamente
- FP permanece fijo durante la ejecución de la función

Estructura de un stack frame



Ejemplo conceptual

```
int f(int x){  
    int y = x + 1;  
    return y;  
}
```

```
int main(){  
    int r = f(3);  
}
```

Durante la llamada:

- PC salta a f
- SP crea nuevo frame
- FP referencia el frame actual

Al retornar:

- SP elimina frame
- PC vuelve a main

Idea clave

PC controla el flujo de ejecución.

SP controla la memoria del stack.

FP permite acceder al contexto de la función.

Juntos permiten:

- llamadas a funciones
- recursión
- manejo de parámetros
- retorno correcto de ejecución

Conectan:

lenguaje $\rightarrow$ runtime $\rightarrow$ arquitectura

Puntero a función

Idea general

Un puntero a función contiene:

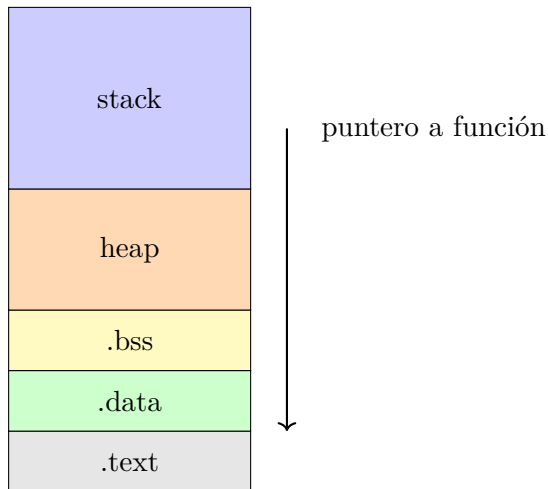
- la dirección de memoria donde comienza el código
- esa dirección pertenece al segmento `.text`
- puede pasarse como parámetro

El puntero se almacena:

- en el stack
- en el heap
- en variables globales (`.data`)

Pero la función siempre está en `.text`

Ubicación en memoria



El puntero está en stack. La función está en .text.

Ejemplo básico

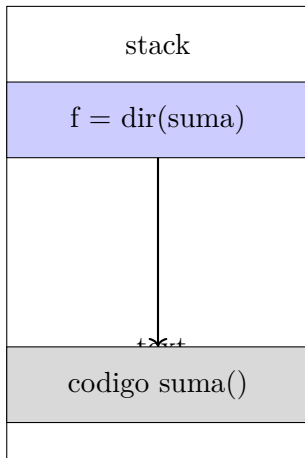
```
int suma(int a, int b)
{
    return a + b;
}

int main()
{
    int (*f)(int,int);

    f = suma;

    int r = f(2,3);
}
```

Qué ocurre en memoria



El puntero almacena la dirección del código.

Pasaje de puntero a función como parámetro

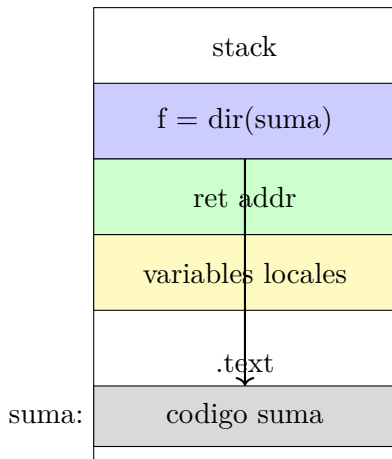
```
int suma(int a,int b)
{
    return a+b;
}

int operar(int (*f)(int,int))
{
    return f(4,5);
}

int main()
{
    int r;

    r = operar(suma);
}
```


Stack frame durante la llamada



El stack frame contiene la dirección de la función.

Callback

```
void aplicar(int (*f)(int))
{
    int r;

    r = f(10);
}

int cuadrado(int x)
{
    return x*x;
}

int main()
{
    aplicar(cuadrado);
}
```

Flujo de memoria

Paso 1:

main guarda dirección de cuadrado

Paso 2:

dirección se copia al stack frame

Paso 3:

llamada indirecta usando esa dirección

Tabla conceptual

elemento	ubicación
código función	.text
puntero función	stack / heap
parámetro	stack
dirección retorno	stack

Relación con polimorfismo

Los punteros a funciones permiten:

- seleccionar comportamiento en ejecución
- implementar estrategias
- simular polimorfismo en C

Ejemplo conceptual:

misma función recibe distintas funciones.

Idea clave

Un puntero a función:

- no copia el código
- copia la dirección del código
- permite llamadas dinámicas
- conecta el stack con el segmento .text

Referencias

-  AHO, ALFRED, SETHI, RAVI, ULLMAN, JEFFREY D., LAM, MONICA S., 2008, *Compiladores. Principios, técnicas y herramientas*, Addison Wesley.
-  SEBESTA, ROBERT W., 2016, *Concepts of Programming Languages*, Pearson.
-  SCOTT, MICHAEL L., 2015, *Programming Language Pragmatics*, Morgan Kaufmann.
-  PATTERSON, DAVID A., HENNESSY, JOHN L., 2014, *Computer Organization and Design: The Hardware/Software Interface (MIPS Edition)*, Morgan Kaufmann.